

Query by Example

Query by Example

Introduction

Usage

Example Matchers

Fluent API

Introduction

This chapter provides an introduction to Query by Example and explains how to use it.

Query by Example (QBE) is a user-friendly querying technique with a simple interface. It allows dynamic query creation and does not require you to write queries that contain field names. In fact, Query by Example does not require you to write queries by using store-specific query languages at all.

NOTE

This chapter explains the core concepts of Query by Example. The information is pulled from the Spring Data Commons module. Depending on your database, String matching support can be limited.

Usage

The Query by Example API consists of four parts:

- **Probe**: The actual example of a domain object with populated fields.
- **ExampleMatcher**: The `ExampleMatcher` carries details on how to match particular fields. It can be reused across multiple Examples.
- **Example**: An `Example` consists of the probe and the `ExampleMatcher`. It is used to create the query.
- **FetchableFluentQuery**: A `FetchableFluentQuery` offers a fluent API, that allows further customization of a query derived from an `Example`. Using the fluent API lets you specify ordering projection and result processing for your query.

Query by Example is well suited for several use cases:

- Querying your data store with a set of static or dynamic constraints.
- Frequent refactoring of the domain objects without worrying about breaking existing queries.
- Working independently of the underlying data store API.

Query by Example also has several limitations:

- No support for nested or grouped property constraints, such as `firstname = ?0` or `(firstname = ?1 and lastname = ?2)`.
- No support for matching collections or maps.
- Store-specific support on string matching. Depending on your databases, String matching can support starts/contains/ends/regex for strings.
- Exact matching for other property types.

Before getting started with Query by Example, you need to have a domain object. To get started, create an interface for your repository, as shown in the following example:

Sample Person object

```
public class Person {  
  
    @Id  
    private String id;  
    private String firstname;  
    private String lastname;  
    private Address address;  
  
    // ... getters and setters omitted  
}
```

JAVA

The preceding example shows a simple domain object. You can use it to create an `Example`. By default, fields having `null` values are ignored, and strings are matched by using the store specific defaults.

NOTE

Inclusion of properties into a Query by Example criteria is based on nullability. Properties using primitive types (`int`, `double`, ...) are always included unless the [ExampleMatcher](#) ignores the property path.

Examples can be built by either using the `of` factory method or by using [ExampleMatcher](#). `Example` is immutable. The following listing shows a simple `Example`:

Example 1. Simple Example

```

Person person = new Person();           1
person.setFirstname("Dave");           2

Example<Person> example = Example.of(person); 3

```

JAVA

- 1 Create a new instance of the domain object.
- 2 Set the properties to query.
- 3 Create the Example.

You can run the example queries by using repositories. To do so, let your repository interface extend `QueryByExampleExecutor<T>`. The following listing shows an excerpt from the `QueryByExampleExecutor` interface:

The QueryByExampleExecutor

```

public interface QueryByExampleExecutor<T> {

    <S extends T> S findOne(Example<S> example);

    <S extends T> Iterable<S> findAll(Example<S> example);

    // ... more functionality omitted.
}

```

JAVA

Example Matchers

Examples are not limited to default settings. You can specify your own defaults for string matching, null handling, and property-specific settings by using the `ExampleMatcher`, as shown in the following example:

Example 2. Example matcher with customized matching

```

Person person = new Person();           1
person.setFirstname("Dave");           2

ExampleMatcher matcher = ExampleMatcher.matching() 3
    .withIgnorePaths("lastname")           4
    .withIncludeNullValues()               5
    .withStringMatcher(StringMatcher.ENDING); 6

Example<Person> example = Example.of(person, matcher); 7

```

JAVA

- 1 Create a new instance of the domain object.
- 2 Set properties.
- 3 Create an `ExampleMatcher` to expect all values to match. It is usable at this stage even without further configuration.
- 4 Construct a new `ExampleMatcher` to ignore the `lastname` property path.
- 5 Construct a new `ExampleMatcher` to ignore the `lastname` property path and to include null values.
- 6 Construct a new `ExampleMatcher` to ignore the `lastname` property path, to include null values, and to perform suffix string matching.
- 7 Create a new `Example` based on the domain object and the configured `ExampleMatcher`.

By default, the `ExampleMatcher` expects all values set on the probe to match. If you want to get results matching any of the predicates defined implicitly, use `ExampleMatcher.matchingAny()`.

You can specify behavior for individual properties (such as "firstname" and "lastname" or, for nested properties, "address.city"). You can tune it with matching options and case sensitivity, as shown in the following example:

Configuring matcher options

```
ExampleMatcher matcher = ExampleMatcher.matching()
    .withMatcher("firstname", endsWith())
    .withMatcher("lastname", startsWith().ignoreCase());
}
```

JAVA

Another way to configure matcher options is to use lambdas (introduced in Java 8). This approach creates a callback that asks the implementor to modify the matcher. You need not return the matcher, because configuration options are held within the matcher instance. The following example shows a matcher that uses lambdas:

Configuring matcher options with lambdas

```
ExampleMatcher matcher = ExampleMatcher.matching()
    .withMatcher("firstname", match -> match.endsWith())
    .withMatcher("firstname", match -> match.startsWith());
}
```

JAVA

Queries created by `Example` use a merged view of the configuration. Default matching settings can be set at the `ExampleMatcher` level, while individual settings can be applied to particular property paths. Settings that are set on `ExampleMatcher` are inherited by property path settings unless they are defined explicitly. Settings on a property patch have higher precedence than default settings. The following table describes the scope of the various `ExampleMatcher` settings:

Table 1. Scope of `ExampleMatcher` settings

Setting	Scope
Null-handling	<code>ExampleMatcher</code>
String matching	<code>ExampleMatcher</code> and property path
Ignoring properties	Property path
Case sensitivity	<code>ExampleMatcher</code> and property path
Value transformation	Property path

Fluent API

`QueryByExampleExecutor` defines fluent query methods for flexible execution of queries based on `Example` instances through `findBy(Example<S> example, Function<FluentQuery.FetchableFluentQuery<S>, R> queryFunction)`.

As with other methods, it executes a query derived from a `Example`. However, the query function allows you to take control over aspects of query execution that you cannot dynamically control otherwise. You do so by invoking the various intermediate and terminal methods of `FetchableFluentQuery`.

Intermediate methods

- `sortBy`: Apply an ordering for your result. Repeated method calls append each `Sort` (note that `page(Pageable)` using a sorted `Pageable` overrides any previous sort order).
- `limit`: Limit the result count.
- `as`: Specify the type to be read or projected to.
- `project`: Limit the queries properties.

Terminal methods

- `first`, `firstValue`: Return the first value. `first` returns an `Optional<T>` or `Optional.empty()` if the query did not yield any result. `firstValue` is its nullable variant without the need to use `Optional`.
- `one`, `oneValue`: Return the one value. `one` returns an `Optional<T>` or `Optional.empty()` if the query did not yield any result. `oneValue` is its nullable variant without the need to use `Optional`. Throws `IncorrectResultSizeDataAccessException` if more than one match found.
- `all`: Return all results as a `List<T>`.

- `page(Pageable)` : Return all results as a `Page<T>` .
- `slice(Pageable)` : Return all results as a `Slice<T>` .
- `scroll(ScrollPosition)` : Use scrolling (offset, keyset) to retrieve results as a `Window<T>` .
- `stream()` : Return a `Stream<T>` to process results lazily. The stream is stateful and must be closed after use.
- `count` and `exists` : Return the count of matching entities or whether any match exists.

NOTE

Intermediate and terminal methods must be invoked within the query function.

Example 3. Use the fluent API to get a projected Page, ordered by Lastname

```
Page<CustomerProjection> page = repository.findBy(example,
    q -> q.as(CustomerProjection.class)
        .page(PageRequest.of(0, 20, Sort.by("lastname"))))
);
```

JAVA

Example 4. Use the fluent API to get the last of potentially many results, ordered by Lastname

```
Optional<Customer> match = repository.findBy(example,
    q -> q.sortBy(Sort.by("lastname").descending())
        .first())
);
```

JAVA

In Spring Data JPA, you can use Query by Example with Repositories, as shown in the following example:

Example 5. Query by Example using a Repository

```
public interface PersonRepository extends JpaRepository<Person, String> { ... }

public class PersonService {

    @Autowired PersonRepository personRepository;

    public List<Person> findPeople(Person probe) {
        return personRepository.findAll(Example.of(probe));
    }
}
```

JAVA

NOTE

NOTE

Currently, only `SingularAttribute` properties can be used for property matching.

The property specifier accepts property names (such as `firstname` and `lastname`). You can navigate by chaining properties together with dots (`address.city`). You can also tune it with matching options and case sensitivity.

The following table shows the various `StringMatcher` options that you can use and the result of using them on a field named `firstname` :

Table 2. *StringMatcher* options

Matching	Logical result
DEFAULT (case-sensitive)	<code>firstname = ?0</code>
DEFAULT (case-insensitive)	<code>LOWER(firstname) = LOWER(?0)</code>
EXACT (case-sensitive)	<code>firstname = ?0</code>
EXACT (case-insensitive)	<code>LOWER(firstname) = LOWER(?0)</code>
STARTING (case-sensitive)	<code>firstname like ?0 + '%'</code>
STARTING (case-insensitive)	<code>LOWER(firstname) like LOWER(?0) + '%'</code>
ENDING (case-sensitive)	<code>firstname like '%' + ?0</code>
ENDING (case-insensitive)	<code>LOWER(firstname) like '%' + LOWER(?0)</code>
CONTAINING (case-sensitive)	<code>firstname like '%' + ?0 + '%'</code>
CONTAINING (case-insensitive)	<code>LOWER(firstname) like '%' + LOWER(?0) + '%'</code>

NOTE

Regex-matching is not supported by JPA.



Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States

and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. “AWS” and “Amazon Web Services” are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.